REAL-TIME MULTIPLAYER GAME NETWORKING ENGINE

UDP BASED NETWORKING ENGINE FOR A REAL-TIME MULTIPLAYER GAME WHERE LOW LATENCY IS CRITICAL.

NAME: DHARANI S

SRN: PES2UG24CS157

SECTION: C

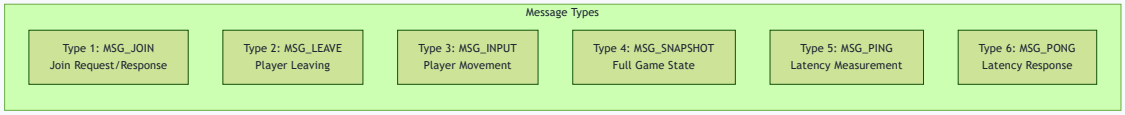
EXPECTATIONS:

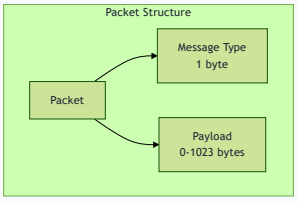
- State synchronization among clients
- Client prediction and correction
- Packet loss tolerance
- Update rate optimization
- Latency and jitter analysis

1. CONNETION ESTABLISHMENT AND SOCKET CREATION:

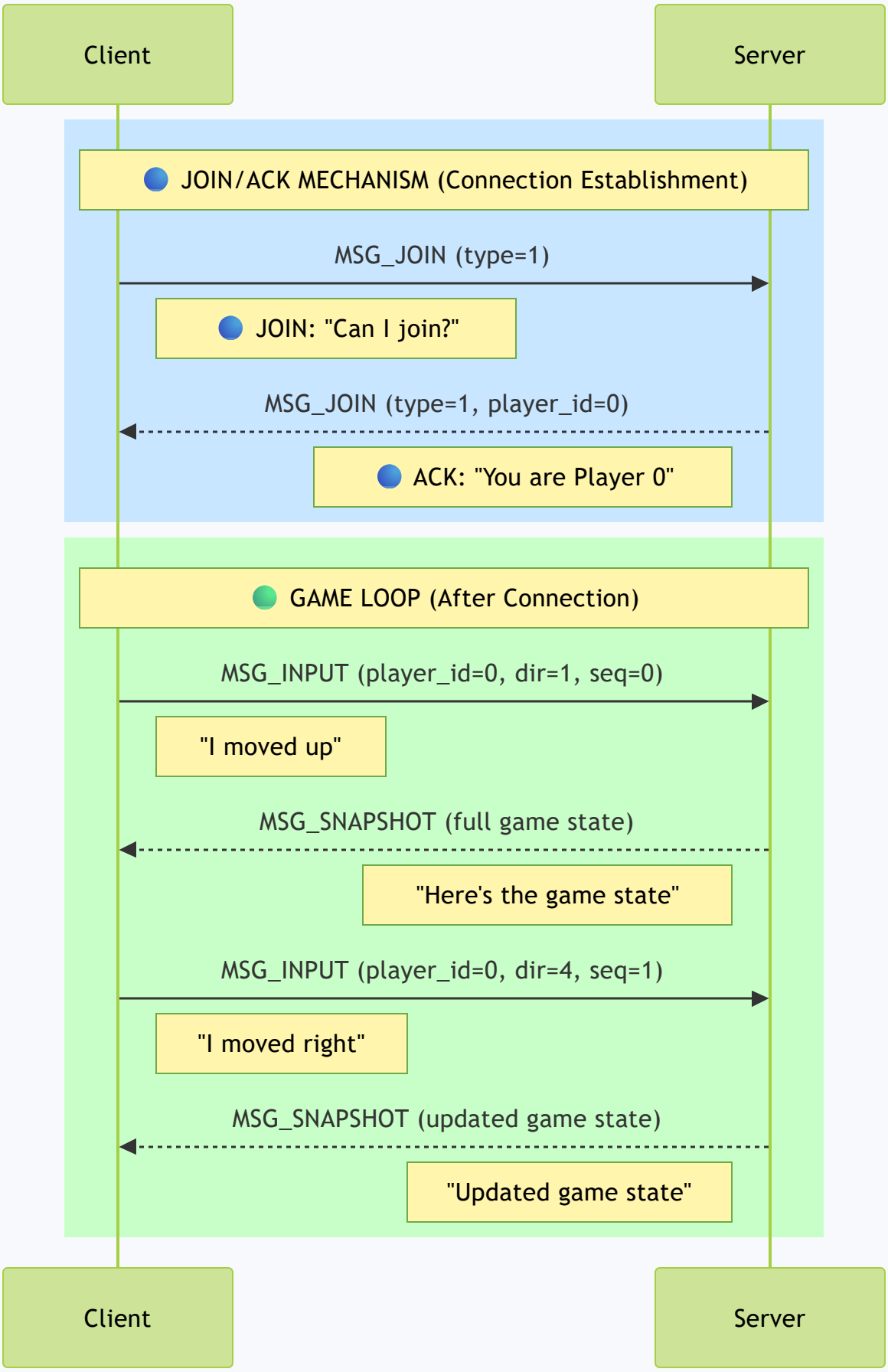
Application layer protocol -> custom udp based protocol

packet structure:





Join - Ack mechanism:



ALL MESSAGE TYPES AND THEIR PAYLOADS

MESSAGE	DIRECTION	MESSAGE TYPE	PAYLOAD
JOIN Request	Client → Server	1	None (0 bytes)
JOIN Response	Server → Client	1	player_id (1 byte)
LEAVE	Client → Server	2	player_id (1 byte)
INPUT	Client → Server	3	player_id, direction, sequence, timestamp (10 bytes)
SNAPSHOT	Server → Client	4	sequence, timestamp, player_count, players[], coins[] (~500 bytes)
PING	Client → Server	5	None or sequence (0–4 bytes)
PONG	Server → Client	6	None or sequence (0–4 bytes)

Code that implements join Ack:

Client side (join):

```
// Send JOIN request
uint8_t join_msg = MSG_JOIN;  // [1]
send_packet(client_sock, &server_addr, &join_msg, 1);
printf("JOIN sent\n");

// Wait for ACK
while (player_id == -1 && !timeout) {
    int len = receive_packet(client_sock, buffer, ...);
    if (buffer[0] == MSG_JOIN && len >= 2) {
        player_id = buffer[1];  // Extract player_id from ACK
        printf("ACK received! I am Player %d\n", player_id);
    }
}
```

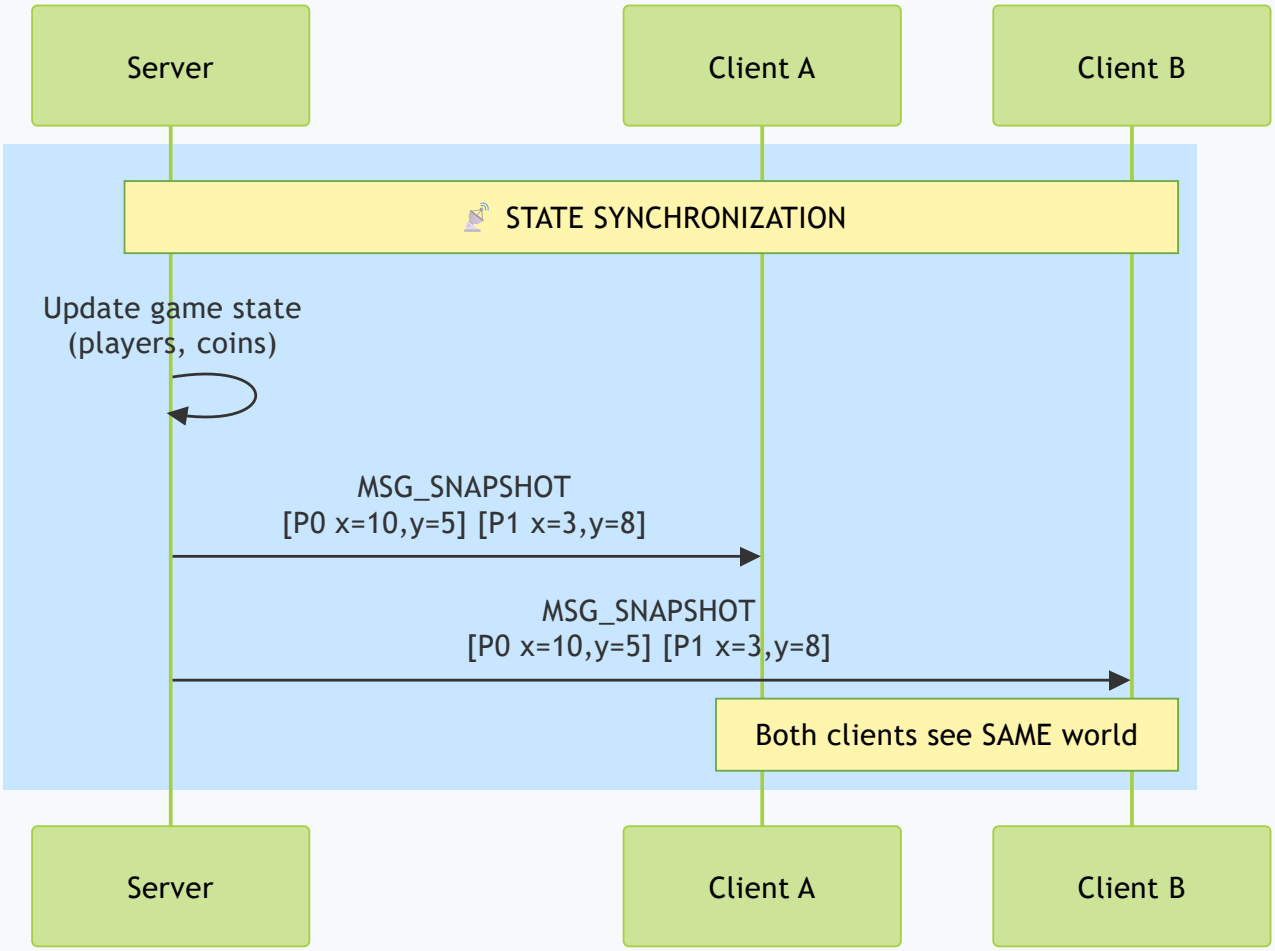
Server- Side (ACK):

```
// Receive JOIN
if (buffer[0] == MSG_JOIN) {
    int id = add_player(&world, &client_addr);

    // Send ACK with player_id
    uint8_t ack[2] = {MSG_JOIN, (uint8_t)id};  // [1, id]
    send_packet(server_sock, &client_addr, ack, 2);
    printf("ACK sent: You are Player %d\n", id);
}
```

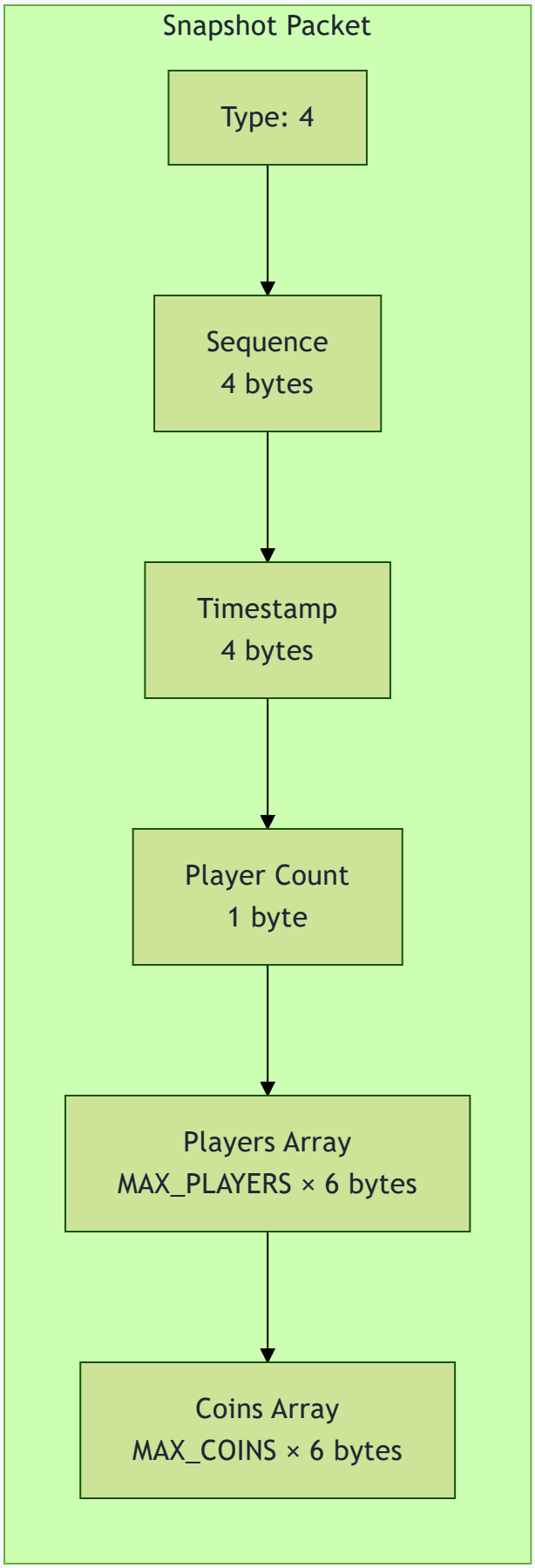
2. STATE SYNCHRONIZATION AMONG CLIENTS

How it Works:

Server broadcasts full game state to all clients at 15Hz. Each client receives and renders the same world.



Packet Structure: MSG_SNAPSHOT (Type 4)



Server Code (Broadcast):

```
void broadcast_snapshot() {
    GameSnapshot snapshot;
    snapshot.sequence = snapshot_sequence++;
    snapshot.timestamp = get_time_ms();
    snapshot.player_count = world.player_count;

    // Copy all players and coins
    memcpy(snapshot.players, world.players, sizeof(world.players));
    memcpy(snapshot.coins, world.coins, sizeof(world.coins));

    // Send to every active player
    for (int i = 0; i < MAX_PLAYERS; i++) {
        if (world.players[i].active) {
            send_packet(server_sock, &world.client_addrs[i],
                        &snapshot, sizeof(snapshot));
        }
    }
}
```

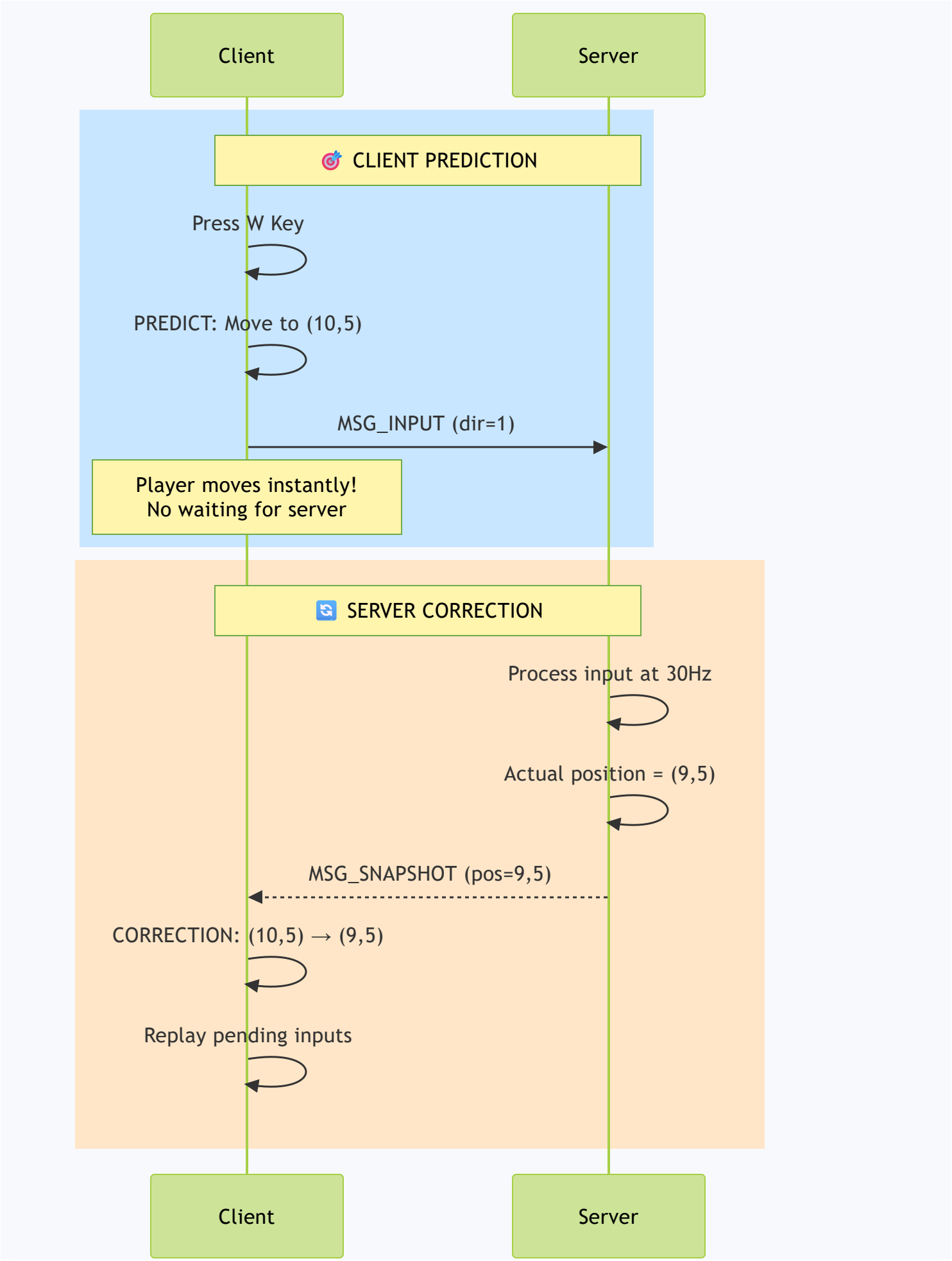
Client Code (Receive):

```
if ((size_t)len >= sizeof(GameSnapshot)) {
    GameSnapshot *snap = (GameSnapshot*)buffer;

    // Update local world with server's authoritative state
    memcpy(world.players, snap->players, sizeof(world.players));
    memcpy(world.coins, snap->coins, sizeof(world.coins));
    world.player_count = snap->player_count;

    // Now all clients have same world state
}
```

3. CLIENT PREDICTION AND CORRECTION

How it Works:

Client moves instantly on key press (prediction). Server later corrects if position was wrong (reconciliation).


Client
Server
🎯 CLIENT PREDICTION
Press W Key
PREDICT: Move to (10,5)
MSG_INPUT (dir=1)
Player moves instantly!
No waiting for server
🔄 SERVER CORRECTION
Process input at 30Hz
Actual position = (9,5)
MSG_SNAPSHOT (pos=9,5)
CORRECTION: (10,5) → (9,5)
Replay pending inputs
Client
Server

Client Code (Prediction):

```
if (dir > 0) {
    InputCommand cmd;
    cmd.player_id = player_id;
    cmd.direction = dir;
    cmd.sequence = input_sequence++;

    // Store for later reconciliation
    input_buffer.inputs[input_buffer.count++] = cmd;

    // PREDICTION: Move instantly!
    update_player(&world, player_id, dir);

    // Send to server
    send_packet_with_stats(out, sizeof(out));
}
```

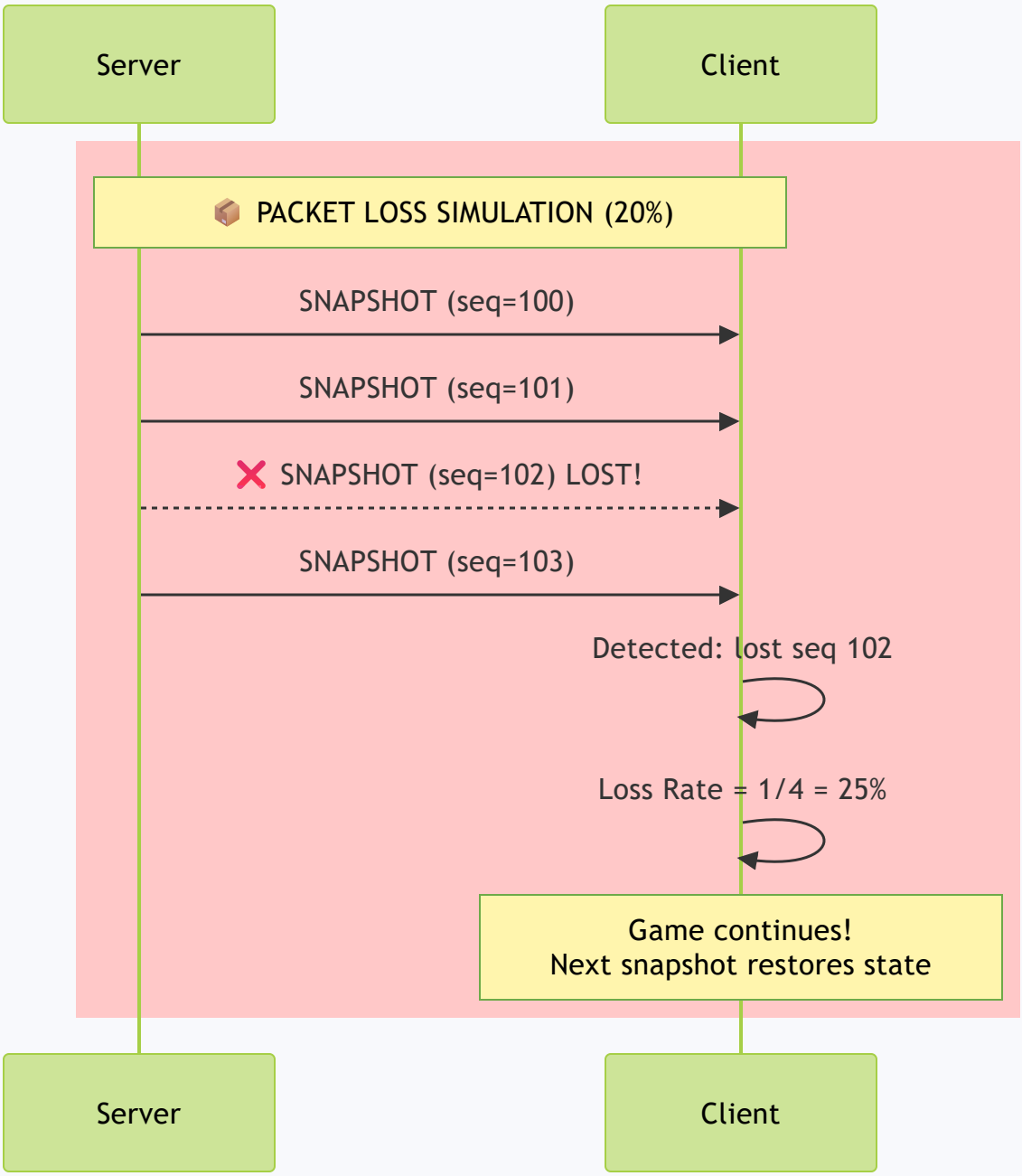
Client Code (Correction):

```
// When snapshot arrives
if (world.players[player_id].x != snap->players[player_id].x ||
    world.players[player_id].y != snap->players[player_id].y) {

    printf("CORRECTION: (%d,%d) -> (%d,%d)\n",
           world.players[player_id].x, world.players[player_id].y,
           snap->players[player_id].x, snap->players[player_id].y);
}

// Server is authority - apply correction
world.players[player_id] = snap->players[player_id];

// Replay pending inputs that server hasn't processed yet
for (int i = 0; i < input_buffer.count; i++) {
    update_player(&world, player_id, input_buffer.inputs[i].direction);
}
```

4. PACKET LOSS TOLERANCE

How it Works:

Sequence numbers detect lost packets. No retransmission - next snapshot contains full state.



Packet Loss Detection:

```
// Track last received sequence
static uint32_t last_sequence = 0;

if (snap->sequence > last_sequence + 1) {
    // Lost packets detected!
    int lost = snap->sequence - last_sequence - 1;
    stats packets_lost += lost;

    printf("PACKET LOSS: lost %d packets (seq %u -> %u)\n",
           lost, last_sequence, snap->sequence);
}
last_sequence = snap->sequence;
```

Simulated Packet Loss (Testing):

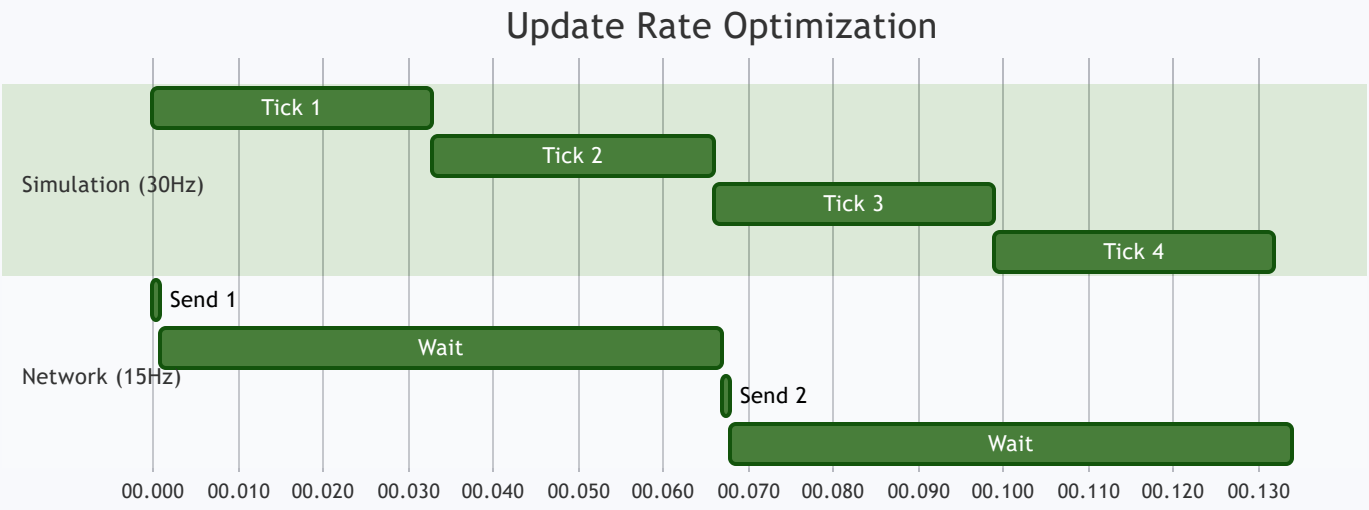
```
void send_packet(int sock, struct sockaddr_in *addr, void *data, size_t len) {
    if (!addr) return;

    // Simulate 20% packet loss for testing
    if (rand() % 100 < 20) {
        return;  // Drop packet
    }

    sendto(sock, data, len, 0, (struct sockaddr *)addr, sizeof(*addr));
}
```

---

5. UPDATE RATE OPTIMIZATION

How it Works:

Simulation runs at 30Hz (accurate physics). Network sends at 15Hz (saves 50% bandwidth).



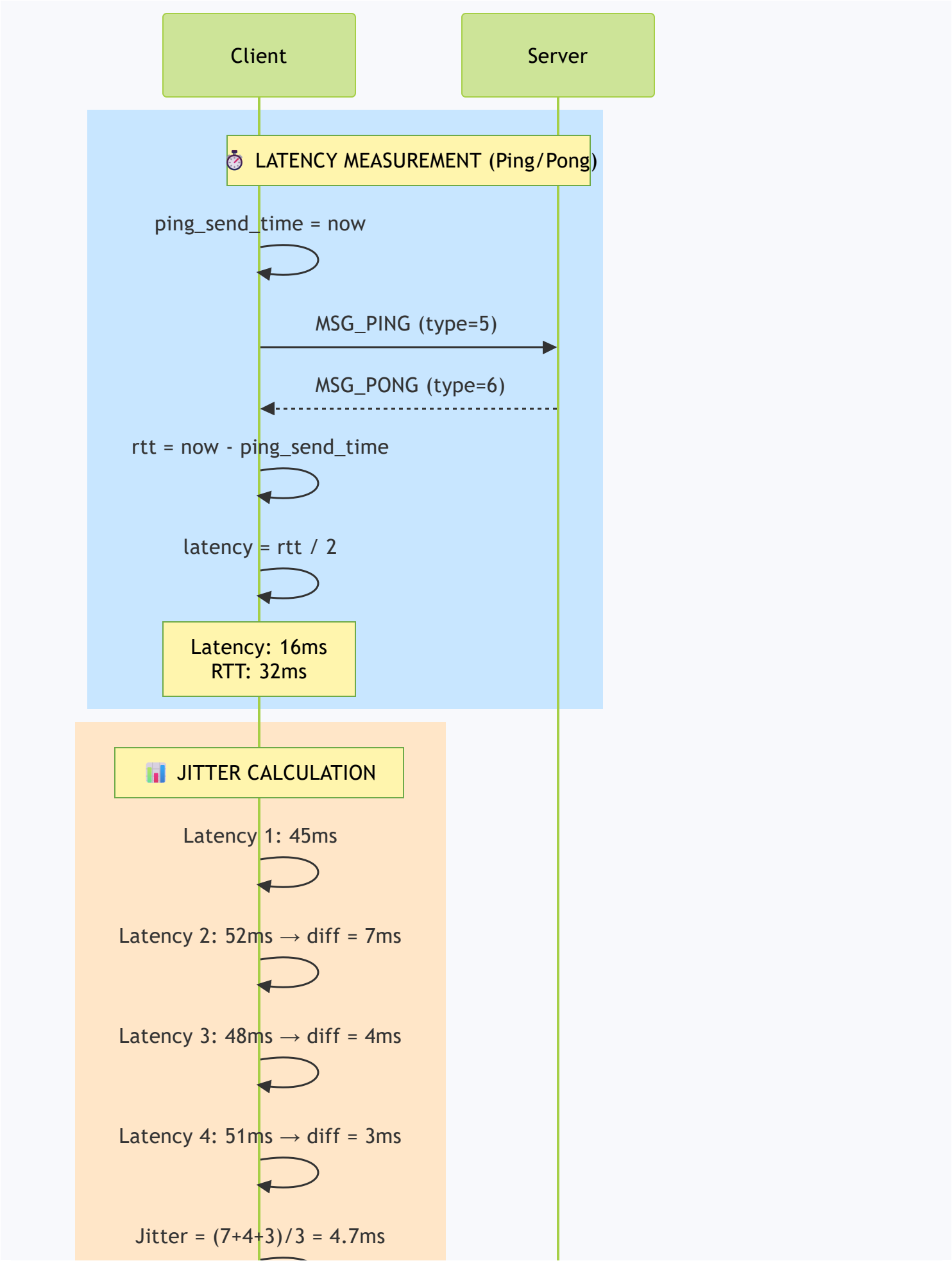
Code Implementation:

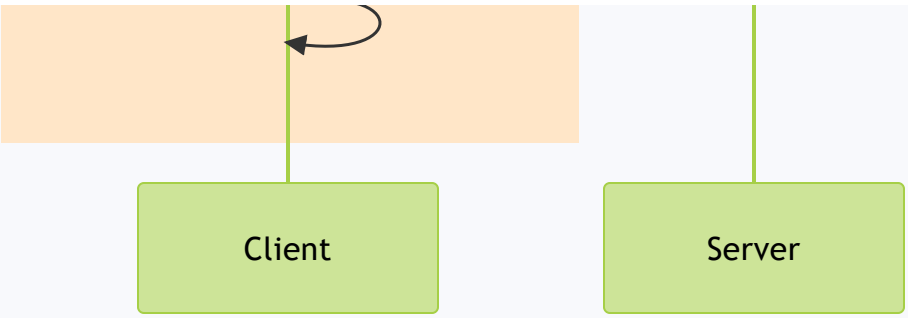
```
#define TICK_RATE 30        // Simulation: 30 times/second
#define NETWORK_SEND_RATE 15 // Network: 15 times/second

while (running) {
    uint32_t now = get_time_ms();

    // Fixed timestep simulation (30Hz)
    if (now - last_tick_time >= (1000 / TICK_RATE)) {
        simulate_fixed_tick(&world);  // Update game logic
        last_tick_time = now;
    }

    // Network broadcast (15Hz) - half the simulation rate
    if (now - last_broadcast >= (1000 / NETWORK_SEND_RATE)) {
        broadcast_snapshot();  // Send state to clients
        last_broadcast = now;
    }
}
```

6. LATENCY AND JITTER ANALYSIS

How it Works:

Ping/Pong messages measure RTT. Sequence tracking detects packet loss. Exponential moving average smooths measurements.


Client
Server
⏱️ LATENCY MEASUREMENT (Ping/Pong)
ping_send_time = now
MSG_PING (type=5)
MSG_PONG (type=6)
rtt = now - ping_send_time
latency = rtt / 2
Latency: 16ms
RTT: 32ms
📊 JITTER CALCULATION
Latency 1: 45ms
Latency 2: 52ms → diff = 7ms
Latency 3: 48ms → diff = 4ms
Latency 4: 51ms → diff = 3ms
Jitter = (7+4+3)/3 = 4.7ms



Latency Measurement (Ping/Pong):

```
// Client sends ping
if (now - last_ping >= 1000) {
    uint8_t ping = MSG_PING;
    ping_send_time = now;
    send_packet_with_stats(&ping, 1);
    last_ping = now;
}

// Server responds immediately
case MSG_PING:
    uint8_t pong = MSG_PONG;
    send_packet(server_sock, &client_addr, &pong, 1);
    break;

// Client receives pong
case MSG_PONG:
    uint32_t rtt = get_time_ms() - ping_send_time;
    uint32_t latency = rtt / 2;  // One-way latency
    break;
```

Jitter Calculation:

```
static uint32_t prev_latency = 0;
float jitter = 0;

// Calculate jitter as latency variation
float diff = abs((int)latency - (int)prev_latency);
jitter = jitter * 0.9 + diff * 0.1;  // Exponential moving average
prev_latency = latency;
```

Packet Loss Rate:

```
// Detect lost packets via sequence gaps
if (snap->sequence > last_sequence + 1) {
    int lost = snap->sequence - last_sequence - 1;
    stats.packets_lost += lost;
}

// Calculate loss percentage
stats.packet_loss_rate = (float)stats.packets_lost /
                         (stats.packets_lost + stats.packets_received) * 100;
```

Displayed Network Stats:

```
Latency: 5 ms | Jitter: 2 ms | Loss: 18% | RTT: 16 ms
```

SUMMARY TABLE

FEATURE	IMPLEMENTATION	KEY CODE
State Sync	Full snapshots at 15Hz	`broadcast_snapshot()`
Prediction	Move instantly on input	`update_player()` before server
Correction	Server state overwrites	`world.players[id] = snap->players[id]`
Loss Tolerance	Sequence numbers, no retransmit	`if (seq > last_seq+1) lost++`
Rate Optimization	Simulate 30Hz, send 15Hz	`TICK_RATE=30, NETWORK_SEND_RATE=15`
Latency	Ping/Pong RTT measurement	`rtt = now - ping_send_time`
Jitter	Latency variance	`jitter = (curr - prev)`